

Robust Optimization: Assignment 2

Benjamin Riedel¹, Davide Carecci²

¹Benjamin Riedel is with the Department of Business Intelligence, Friedrich Schiller Universität Jena

²Davide Carecci is with the Department of Electronics, Information and Bioengineering, Politecnico di Milano

I. IMPLEMENTING COLUMN-AND-CONSTRAINT GENERATION: EXERCISE 4.3

Visit the link of COLAB for Exercise 4.3: Exercises 4.3

The iterative algorithm was implemented applying the following logic:

A. Take any $\hat{x} \in X$

Taking $\hat{x}$ as a vector of zeros is feasible because of the lack of restrictions other than binary constraint. A more reasonable choice would be the solution of the deterministic model, which was calculated already. If the deterministic Model has not been solved opening no facility is feasible as well.

B. Finding the worst-case for $\hat{x}$

By solving problem (4.16) we identify $\hat{z}$. For that we construct the matrix form of **problem 4.18**. To do that the matrix containing the y_{ij} was reshaped in multiple row vectors y_n in $\mathbb{R}^n$. Resulting in a $m \times n$ vector.

The resulting a_2 matrix (corresponding to y) is of shape $(m+n+n \cdot m \times n \cdot m)$.

$$\begin{bmatrix} [1 & 0 & \dots & 0] & [1 & 0 & \dots & 0] & \dots & [1 & 0 & \dots & 0] \\ [0 & 1 & 0 & \dots & 0] & [0 & 1 & 0 & \dots & 0] & \dots & [0 & 1 & 0 & \dots & 0] \\ & & \ddots & & m-2 \text{ more rows} & & \\ [1 & 1 & \dots & 1] & \mathbf{0}(m)^T & \dots & \mathbf{0}(m)^T \\ & \mathbf{0}(m)^T & & [1 & 1 & \dots & 1] & \mathbf{0}(m)^T & \dots & \mathbf{0}(m)^T \\ & & \ddots & & n-2 \text{ more rows} & & \\ & & & & -\mathbf{I}(n \cdot m) & & \end{bmatrix}$$

With $\mathbf{0}(m)^T$ being a row vector of zeros of dimension m and $\mathbf{I}(n \cdot m)$ the identity matrix with $n \cdot m$ rows and columns.

The first m rows define the demand restrictions, the next n the capacity constraints for the production in the opened facilities and the last $m \cdot n$ the not negativity constraints.

c_2 is defined through reshaping the $m \times n$ matrix of $(r-d)$ into a vector of dimension $m \cdot n$.

The a_1 matrix is of size $m+n+m \cdot n \times n$. Since x only appears in n restrictions only in rows $m+1$ to $m+n$ there are non zeros.

$$\begin{bmatrix} \mathbf{0}(n)^T & & & \\ & \ddots & & m-1 \text{ more rows} \\ [-P_1 & 0 & \dots & 0] \\ [0 & -P_2 & 0 & \dots & 0] \\ & \ddots & & n-2 \text{ more rows} \\ & -\mathbf{0}(n \cdot m \times n) & & \end{bmatrix}$$

With that it is easier to follow the generalized structure of problem **4.12**. The latter formulation was then useful to derive the KKT conditions as in **Corollary 4.9**. The *findworstcase* function was employed ($\hat{x}$ is an input).

C. Iterate

The problem was reformulated according to problem **4.14**. The *subproblem* function was employed. Note that an additional constraint was written for each additional vertex of the input uncertainty set. In the last cell of the COLAB file, the two functions work together in a *while* loop; the loop condition is the objective function value of the *findworstcase* problem being approximately ($\pm$ tolerance) equal to the one of the *subproblem*, for the same $\hat{x}$.

II. COMPARISON OF APPROXIMATE WORST-CASE BOUNDS: EXERCISE 5.4-5

Visit the link of COLAB for Exercises 5.4-5: Exercises 5.4-5

The same structure for the implementation of the exact problem solution was proposed as for *Exercise 4.3*.

A. Exercise 5.4

Later, the exact solution result was compared to the other approximated implementations, for different dimensions of the budgeted set.

- $\Gamma = 0$. No uncertainty was considered. Consistently, every version of the problem returns a solution equal to the one of the deterministic problem.
- $\Gamma = 1$. The static robust counterpart (SRC) was more conservative than the other problem implementations. When y_{ij} can be adapted as second-stage decisions, the **worst-case** profit increased. Both AARC and L-AARC were good approximations of the exact solution.
- $\Gamma = 4$. The static robust counterpart (SRC) was more conservative than the other problem implementations. L-AARC was a good approximations of the exact solution. Affine mapping only (AARC) was not able to fully grasp the possible adaptation of y_{ij} , thus it ended up being more conservative. Due to a higher radius of the designed uncertainty set, the profit values returned by the adaptable problem implementations is lower with respect to the case of lower Γ .
- $\Gamma = 11$. Very conservative design of the uncertainty set. No adaptation of y_{ij} could help to improve the profits, since $\hat{x}$ and $\hat{z}$ were selected such to cover very pessimistic demand scenarios.

B. Exercise 5.5

Similarly, the true worst-case solution result associated to the first-stage decision was compared for the different approximated implementations and for different dimensions of the budgeted set.

- $\Gamma = 0$. Actual worst-case profits were coherent with the values obtained by solving the robust problem for all the problem implementations, because of no uncertainty.
- $\Gamma = 1$. The same $\hat{x}$ solution found by the SRC implementation can return a much higher worst-case profit when y_{ij} are made adjustable (+142%).
- $\Gamma = 4$. The same $\hat{x}$ solution found by the SRC implementation can return a higher worst-case profit when y_{ij} are made adjustable (+51.8%).
- $\Gamma = 11$. Actual worst-case profits were coherent with the values obtained by solving the robust problem for all the problem implementations.

In conclusion, in the case of problem **4.18**, the L-AARC problem formulation is a good approximation of the exact solution of a TSARC problem (**4.12**), as it returns the same value of the solution calculated by mean of the column and constraint algorithm.